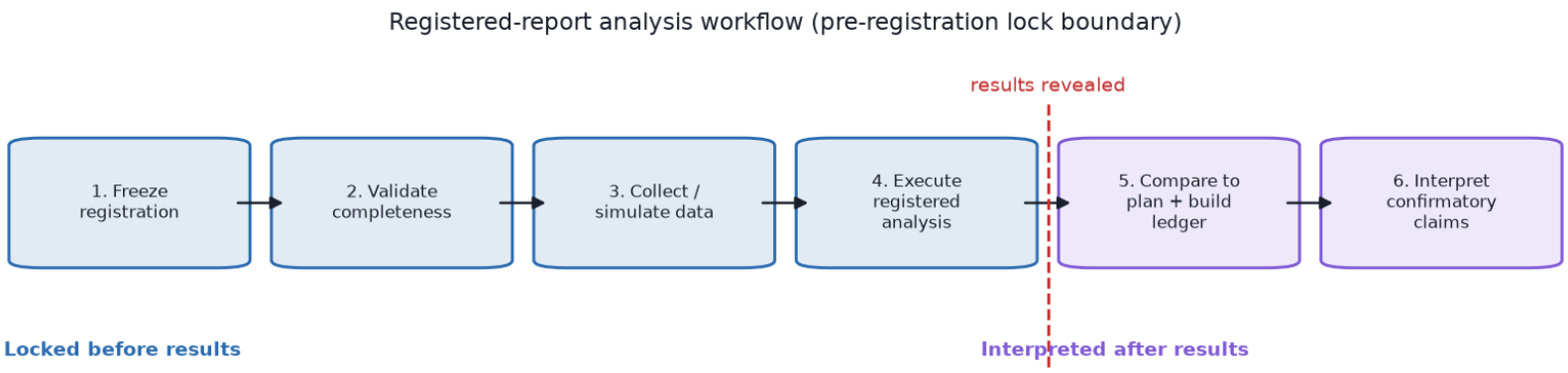
Methods and analysis plan

Methods and analysis plan I

The method is the registered-report workflow itself, implemented as tested code. It has four locked stages that complete before any result is observed, followed by two interpretation stages that run only after the deviation ledger is built. [@fig:analysis_dag] shows this ordering and the lock boundary it protects.



Methods and analysis plan II

Figure 1: Registered-report analysis workflow, rendered from `analysis_plan_stages()` in `src/registered_report/demo_study.py`. The four blue stages — freeze registration, validate completeness, collect/simulate data, and execute the registered analysis — lock before results are revealed (dashed boundary). The two purple stages — compare to plan and build the deviation ledger, then interpret confirmatory claims — run only afterwards. The `locks_before_results` flag on each stage is what enforces the boundary in code.

Frozen registration I

`freeze_registration` deep-copies the registration, strips any pre-existing hash, and stamps a canonical SHA-256 hash over the sorted-key JSON encoding. `validate_registration` then checks that `title`, `version`, `hypotheses`, `outcomes`, `exclusion_rules`, and `analysis_plan` are present; that each hypothesis has a unique `id` and a `claim`; that each outcome names a `measure` and `analysis`; that `analysis_plan.primary_model` and `seed` are set; and that registered-report stage and ethics-review metadata are supplied. For the demonstration registration this validation returns no findings.

Registered analysis plan I

The analysis plan is fixed before execution:

- **Primary model** — a two-sided label-permutation test on the group mean difference.
- **Significance threshold** — `alpha = 0.05`.
- **Seed** — 20260709, used for both data synthesis and the permutation schedule so the result is reproducible.
- **Permutations** — 2000 label shuffles; the two-sided p-value uses the add-one correction `(k + 1) / (permutations + 1)` and is therefore strictly positive.

Deterministic demonstration data I

Because this is a template, the "data collection" stage is a seeded synthetic generator (`generate_demo_dataset`), **not** a real study. It draws `n = 24` control observations from `Normal(0, 1)` and `n = 24` treatment observations from `Normal(0.8, 1)` using a single seeded generator, so the entire dataset is a pure function of the plan seed. `run_registered_analysis` reads the seed and `alpha` back out of the frozen plan and executes the registered permutation test against this dataset, guaranteeing the executed analysis honours the locked plan. The executed summary is written to `output/data/demo_analysis.json` and is the single source of truth for every number in [Section @sec:results].

Forks replace this generator with genuine data collection while keeping the same `run_registered_analysis` contract, so the confirmatory analysis remains bound to the frozen plan rather than to post-hoc choices.